

Here's a **clear, structured comparison + implementation guide** for **OAuth2, JWT, and Spring Security** with Java examples.

□ 1. What They Are (Conceptual Difference)

Feature	OAuth2	JWT	Spring Security
Type	Authorization Framework	Token Format	Security Framework
Purpose	Delegated access (login via Google, etc.)	Secure data transmission	Secure Java apps
Works With	Access tokens	Self-contained tokens	Authentication + Authorization
State	Usually stateful	Stateless	Both supported
Example	"Login with Google"	Bearer token in API	Role-based security

□ 2. Key Similarities

Similarity	Explanation
Used in Authentication	All can be part of login systems
Used in Authorization	Control access to APIs/resources
Token-based	OAuth2 uses tokens, JWT is a token
Used in APIs	Common in REST microservices
Can be combined	OAuth2 + JWT + Spring Security together

□ 3. Key Differences

Feature	OAuth2	JWT	Spring Security
Role	Protocol	Token	Framework
Dependency	Uses tokens	Independent	Uses both

	(JWT or opaque)		
Storage	Server or external	Client-side	Configurable
Complexity	High	Medium	Medium
Use Case	Third-party login	Stateless API auth	Full security system

❑ 4. How They Work Together

❑ Real-world flow:

1. User logs in via OAuth2 (Google, GitHub)
 2. Server generates JWT token
 3. Client sends JWT in every request
 4. Spring Security validates JWT
-

❑ 5. Java Implementation Examples

✔ 5.1 JWT Implementation (Spring Boot)

❑ Maven Dependency

```
<dependency>
  <groupId>io.jsonwebtoken</groupId>
  <artifactId>jjwt</artifactId>
  <version>0.9.1</version>
</dependency>
```

❑ JWT Utility Class

```
import io.jsonwebtoken.*;
import java.util.Date;

public class JwtUtil {

    private String secret = "mySecretKey";

    public String generateToken(String username) {
        return Jwts.builder()
            .setSubject(username)
            .setIssuedAt(new Date())
```

```

        .setExpiration(new Date(System.currentTimeMillis() + 1000 * 60 * 60))
        .signWith(SignatureAlgorithm.HS256, secret)
        .compact();
    }

    public String extractUsername(String token) {
        return Jwts.parser()
            .setSigningKey(secret)
            .parseClaimsJws(token)
            .getBody()
            .getSubject();
    }
}

```

❑ JWT Filter

```

import javax.servlet.*;
import javax.servlet.http.*;
import java.io.IOException;

public class JwtFilter extends GenericFilter {

    @Override
    public void doFilter(ServletRequest request, ServletResponse response, FilterChain chain)
        throws IOException, ServletException {

        HttpServletRequest req = (HttpServletRequest) request;
        String authHeader = req.getHeader("Authorization");

        if (authHeader != null && authHeader.startsWith("Bearer ")) {
            String token = authHeader.substring(7);
            // validate token logic
        }

        chain.doFilter(request, response);
    }
}

```

✔ 5.2 Spring Security Configuration

```

import org.springframework.context.annotation.*;
import org.springframework.security.config.annotation.web.builders.HttpSecurity;
import org.springframework.security.web.SecurityFilterChain;

@Configuration
public class SecurityConfig {

```

```

@Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
    http
        .csrf().disable()
        .authorizeHttpRequests(auth -> auth
            .requestMatchers("/public").permitAll()
            .anyRequest().authenticated()
        )
        .httpBasic();

    return http.build();
}
}

```

✔ 5.3 OAuth2 Login (Google Example)

□ Dependency

```

<dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-starter-oauth2-client</artifactId>
</dependency>

```

□ application.yml

```

spring:
  security:
    oauth2:
      client:
        registration:
          google:
            client-id: YOUR_CLIENT_ID
            client-secret: YOUR_SECRET
            scope:
              - email
              - profile

```

□ Security Config for OAuth2

```

@Configuration
public class OAuth2SecurityConfig {

    @Bean
    public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
        http
            .authorizeHttpRequests(auth -> auth

```

```

        .anyRequest().authenticated()
    )
    .oauth2Login();

    return http.build();
}
}

```

□ 6. Combined Example (OAuth2 + JWT + Spring Security)

```

@RestController
public class AuthController {

    @Autowired
    private JwtUtil jwtUtil;

    @GetMapping("/login-success")
    public String loginSuccess(Authentication auth) {
        String username = auth.getName();
        return jwtUtil.generateToken(username);
    }
}

```

□ 7. When to Use What

Scenario	Use
Login with Google/Facebook	OAuth2
Stateless REST API	JWT
Full Java security control	Spring Security
Microservices	JWT + Spring Security
Enterprise apps	OAuth2 + JWT + Spring Security

□ 8. Real-Time Architecture

```

Client → OAuth2 Login → Server
      ↓
    Generate JWT
      ↓
Client → Send JWT → Spring Security → Validate → API Access

```

□ 9. Summary

- **OAuth2** → "Who are you?" (Authorization via providers)
 - **JWT** → "Proof of identity" (Token)
 - **Spring Security** → "Security guard" (Controls access)
-

If you want next level □

I can give you:

- ✓ **Full Spring Boot project (OAuth2 + JWT + MySQL + React)**
- ✓ **Microservices architecture with API Gateway + JWT**
- ✓ **Interview questions (FAANG level)**